

Category	Description	Popular tools
Data cleaning and annotation	Before training a model, data must be cleaned and properly labelled. This ensures accuracy and reduces biases in machine learning models. Annotation tools help in tagging datasets for supervised learning.	Prodigy (AI-assisted annotation), Label Studio (open source data labelling), Snorkel (programmatic data labelling)
Dataset management	Keeping track of dataset versions, changes, and metadata is crucial for reproducibility in ML projects. Dataset management tools help maintain consistency and collaboration.	Weights & Biases (W&B) (experiment tracking and dataset versioning), DVC (Data Version Control, Git-like versioning for datasets)
Synthetic data generation	When real-world data is insufficient or lacks diversity, synthetic data generation can create realistic, model-ready datasets. AI-powered tools can generate high-quality training data for various applications.	GPT-based generators (AI-generated synthetic data), Other tools (custom solutions for domain-specific needs)

Table 4: Tools for preprocessing and managing training data

trained model.

- Data privacy or customisation is required, and external APIs are not an option.
- Improving efficiency by training a smaller model on targeted data instead of using a massive model.

Frameworks for fine-tuning

Several frameworks facilitate fine-tuning LLMs efficiently.

Hugging Face Trainer: The trainer API from Hugging Face simplifies the training process.

```
from transformers import Trainer, TrainingArguments,
AutoModelForSequenceClassification, AutoTokenizer
from datasets import load_dataset

# Load dataset and model
dataset = load_dataset("imdb")
model = AutoModelForSequenceClassification.from_pretrained("bert-base-uncased", num_labels=2)
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")

def preprocess_data(examples):
    return tokenizer(examples["text"], truncation=True, padding=True)

dataset = dataset.map(preprocess_data, batched=True)

# Define training arguments
training_args = TrainingArguments(
    output_dir="./results", per_device_train_batch_size=8,
    num_train_epochs=3, logging_dir="./logs"
)

# Trainer instance
trainer = Trainer(
    model=model, args=training_args, train_
```

```
dataset=dataset["train"], eval_dataset=dataset["test"]
)
```

```
trainer.train()
```

PyTorch Lightning: PyTorch Lightning offers a structured approach for scalable model training.

```
import pytorch_lightning as pl
from transformers import AutoModel, AutoTokenizer
class LLMFineTuner(pl.LightningModule):
    def __init__(self, model_name="bert-base-uncased", num_labels=2):
        super().__init__()
        self.model = AutoModel.from_pretrained(model_name, num_labels=num_labels)

    def forward(self, x):
        return self.model(x)
```

Distributed training

For large scale fine-tuning, distributed training tools help efficiently utilise multiple GPUs or TPUs.

- **DeepSpeed:** Optimises large scale training with memory-efficient techniques.
- **Horovod:** Open source framework for distributed deep learning.
- **Ray Train:** Provides scalable model training across clusters.

Here's an example for enabling DeepSpeed in Hugging Face Trainer:

```
training_args = TrainingArguments(
    output_dir="./results",
    per_device_train_batch_size=8,
    num_train_epochs=3,
    logging_dir="./logs",
```